OAVS Computer Teacher (TGT) Exam

Complete MCQ Practice Book

TOPIC: DATA TYPES IN C PROGRAMMING

150+ Questions | With Explanations | 2 Practice Tests

Topics Covered	int, char, float, double Basic, Derived, User-Defined Types Range & Size of Types Type Conversion & Casting sizeof() Operator
Total Questions	150+ MCQs (with explanations)
Practice Tests	2 Full-Length Tests (20 Qs each) + Answer Keys
Reference Tables	Complete Data Type Size & Range Chart
Exam Relevance	OAVS TGT Computer Science 2024-25

Prepared for: OAVS Computer Teacher (TGT) Aspirants | C Programming — Data Types

TABLE OF CONTENTS

Cover & Overview	1
Quick Reference: Data Type Sizes & Ranges	3
Chapter 1: Introduction to Data Types in C	5
Chapter 2: Integer Data Types	6
Chapter 3: char Data Type	7
Chapter 4: float and double	8
Chapter 5: sizeof() Operator	9
Chapter 6: Type Conversion & Type Casting	10
Chapter 7: Derived & User-Defined Types	11
Chapter 8: Output-Based & Tricky Questions	12
Chapter 9: Range, Size Reference & Type Modifiers	13
Practice Test 1 — Quick Fire (20 Qs)	—
Practice Test 2 — Output & Concepts (20 Qs)	—
Answer Keys	—

Quick Reference Table

Data Type Sizes, Ranges & Format Specifiers

This table summarises all C data types. Memorise sizes and format specifiers for the exam.

Data Type	Size (bytes)	Format Spec	Range / Notes
char	1	%c / %d	-128 to 127 (signed)
unsigned char	1	%c / %u	0 to 255
short int	2	%hd	-32,768 to 32,767
unsigned short	2	%hu	0 to 65,535
int	4	%d	-2,147,483,648 to 2,147,483,647
unsigned int	4	%u	0 to 4,294,967,295
long int	4 / 8*	%ld	-2^31 to 2^31-1 (32-bit)
unsigned long	4 / 8*	%lu	0 to 2^32-1 (32-bit)
long long int	8	%lld	-9.2×10^18 to 9.2×10^18
unsigned long long	8	%llu	0 to 1.8×10^19
float	4	%f	±3.4×10^38, ~6-7 sig digits
double	8	%lf / %f	±1.7×10^308, ~15-16 digits
long double	10-16*	%Lf	±1.18×10^4932 (x86 80-bit)
void	—	—	Incomplete / no value type
pointer (32-bit)	4	%p	Memory address
pointer (64-bit)	8	%p	Memory address

** Platform-dependent values. Values shown for typical 32-bit systems unless noted.*

Chapter 1: Introduction to Data Types in C

Basic Concepts & Classification

Q1. Which of the following is NOT a basic data type in C?

A) int
B) float
C) string
D) char

✔ **Answer: (C)**

Explanation: C does not have a built-in 'string' data type. Strings in C are handled as arrays of char. int, float, and char are all primitive/basic data types in C.

Q2. Which of the following best classifies data types in C?

A) Primary and Secondary
B) Basic, Derived, and User-Defined
C) Integer and Non-Integer
D) Static and Dynamic

✔ **Answer: (B)**

Explanation: C classifies data types into three categories: Basic (int, char, float, double), Derived (arrays, pointers, functions), and User-Defined (struct, union, enum, typedef).

Q3. What is the size of an 'int' data type on a 32-bit compiler?

A) 2 bytes
B) 4 bytes
C) 8 bytes
D) 1 byte

✔ **Answer: (B)**

Explanation: On a 32-bit compiler (like GCC on 32-bit systems), an 'int' occupies 4 bytes (32 bits). On older 16-bit compilers, it was 2 bytes.

Q4. Which header file is required to use INT_MAX and INT_MIN constants?

A) stdio.h
B) stdlib.h
C) limits.h
D) math.h

✔ **Answer: (C)**

Explanation: defines macros like INT_MAX, INT_MIN, CHAR_MAX, etc., which specify the range limits of data types.

Q5. The keyword 'void' in C represents:

A) An empty data type
B) A zero value
C) A null pointer
D) An undefined variable

✔ **Answer: (A)**

Explanation: 'void' represents an empty type. It indicates the absence of a value. It is used for functions that return nothing and for generic pointers (void*).

Q6. Which of the following is a derived data type in C?

A) int
B) float
C) array
D) enum

✔ **Answer: (C)**

Explanation: Array is a derived data type in C. Derived types are built from basic/primary types. int and float are basic types; enum is a user-defined type.

Q7. Which of the following is a user-defined data type in C?

A) double
B) pointer
C) struct
D) char

✔ **Answer: (C)**

Explanation: 'struct' (structure) is a user-defined data type that allows grouping variables of different types under one name. double, char are basic; pointer is derived.

Q8. What does the format specifier '%d' represent in C?

A) double
B) decimal integer
C) digit count
D) data type

✔ **Answer: (B)**

Explanation: '%d' is the format specifier for signed decimal integer (int). It is used with printf() and scanf() to handle integer values.

Q9. Which data type is used to store a single character in C?

A) string
B) char
C) wchar
D) letter

✔ **Answer: (B)**

Explanation: 'char' is used to store a single character. It occupies 1 byte of memory and can store any ASCII character.

Q10. In C, what is the range of a signed char?

A) 0 to 255
B) -128 to 127
C) -256 to 255
D) 0 to 127

✔ **Answer: (B)**

Explanation: A signed char uses 1 byte (8 bits). With the sign bit, it ranges from -128 to +127 (i.e., -2^7 to 2^7 - 1).

Chapter 2: Integer Data Types

int, short, long, unsigned & their Ranges

Q11. What is the range of 'unsigned int' on a 32-bit system?

A) -2,147,483,648 to 2,147,483,647
B) 0 to 4,294,967,295
C) 0 to 65,535
D) 0 to 2,147,483,647

✔ **Answer: (B)**

Explanation: An unsigned int on a 32-bit system uses all 32 bits for the magnitude (no sign bit), giving a range of 0 to 2^32 - 1 = 4,294,967,295.

Q12. What is the size of 'short int' in C?

A) 1 byte
B) 2 bytes
C) 4 bytes
D) 8 bytes

✔ **Answer: (B)**

Explanation: 'short int' (or simply 'short') is guaranteed to be at least 16 bits (2 bytes). Its range is -32,768 to 32,767 for signed short.

Q13. What is the output of the following code?

```
#include
int main(){
int x = 32767;
x++;
printf("%d", x);
return 0;
}
```

A) 32768
B) -32768
C) 0
D) Undefined behavior on all systems

✔ **Answer: (D)**

Explanation: On a system where int is 16 bits, x++ would overflow to -32768 (signed overflow is UB in C). On 32-bit systems, int holds more and x becomes 32768 normally. The correct answer is technically D because signed overflow is undefined behavior per C standard.

Q14. Which format specifier is used for 'long int' in printf()?

A) %d
B) %ld
C) %li
D) %L

✔ **Answer: (B)**

Explanation: '%ld' is the correct format specifier for long int. Using '%d' for a long int may produce incorrect output on some platforms.

Q15. What is the size of 'long long int' in C (on most 64-bit systems)?

A) 4 bytes

B) 6 bytes
C) 8 bytes
D) 16 bytes

✔ **Answer: (C)**

Explanation: 'long long int' is guaranteed to be at least 64 bits (8 bytes) by the C99 standard. Its range is -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.

Q16. What will sizeof(int) return on a typical 64-bit Linux system?

A) 2
B) 4
C) 8
D) 16

✔ **Answer: (B)**

Explanation: Even on 64-bit Linux systems, sizeof(int) typically returns 4 bytes. The LP64 data model used by Linux keeps int at 32 bits.

Q17. Which of the following declarations is INVALID in C?

A) unsigned int a;
B) signed char b;
C) unsigned float c;
D) long long int d;

✔ **Answer: (C)**

Explanation: 'unsigned float' is invalid in C. The 'unsigned' qualifier can only be applied to integer types (char, short, int, long). Floating-point types are always signed.

Q18. What is the output?

```
#include
int main(){
unsigned int x = -1;
printf("%u", x);
return 0;
}
```

A) -1
B) 0
C) 4294967295
D) Compile error

✔ **Answer: (C)**

Explanation: Assigning -1 to an unsigned int wraps around. On a 32-bit system, -1 in two's complement is all 1s, which equals 4,294,967,295 as an unsigned value.

Q19. The range of 'signed int' on a 32-bit system is:

A) 0 to 4,294,967,295
B) -2,147,483,648 to 2,147,483,647
C) -32,768 to 32,767
D) -1,073,741,824 to 1,073,741,823

✔ **Answer: (B)**

Explanation: Signed int on 32 bits: range = -2^31 to 2^31 - 1 = -2,147,483,648 to 2,147,483,647.

Q20. Which keyword makes a variable store only non-negative integers?

A) positive
B) signed

C) unsigned
D) natural

✔ **Answer: (C)**

Explanation: The 'unsigned' keyword restricts a variable to store only non-negative integers (0 and positive). All bits are used for magnitude.

Q21. What is the range of 'unsigned short int'?

A) -32768 to 32767
B) 0 to 65535
C) 0 to 32767
D) -65536 to 65535

✔ **Answer: (B)**

Explanation: Unsigned short uses 16 bits: range = 0 to 2^16 - 1 = 0 to 65,535.

Q22. Which of the following correctly declares a long long integer variable?

A) longlong int x;
B) long long x;
C) long_long x;
D) Both A and B

✔ **Answer: (B)**

Explanation: 'long long x;' is the correct declaration. 'long long int x;' is also valid. 'longlong' and 'long_long' are not valid C keywords.

Q23. What format specifier is used for unsigned long long int?

A) %llu
B) %ull
C) %Lu
D) %uLL

✔ **Answer: (A)**

Explanation: '%llu' is the correct format specifier for unsigned long long int. 'llu' stands for long long unsigned.

Q24. In C, the size of 'int' is:

A) Always 2 bytes
B) Always 4 bytes
C) Implementation-defined, at least 16 bits
D) Always 8 bytes

✔ **Answer: (C)**

Explanation: The C standard does not fix the size of int. It only guarantees at least 16 bits. The actual size is implementation-defined (usually 4 bytes on 32/64-bit systems).

Q25. What is the output?

```
#include
int main(){
int a = 2147483647;
printf("%d", a+1);
return 0;
}
```

A) 2147483648
B) -2147483648
C) 0
D) Undefined behavior

✔ **Answer: (D)**

Explanation: INT_MAX + 1 causes signed integer overflow, which is undefined behavior in C. In practice on most systems it wraps to -2147483648, but the standard says it is UB.

Chapter 3: char Data Type

Characters, ASCII, and Character Operations

Q26. What is the size of the 'char' data type in C?

A) 2 bytes
B) 4 bytes
C) 1 byte
D) 8 bits (always)

✔ **Answer: (C)**

Explanation: The C standard guarantees sizeof(char) == 1. A char is exactly 1 byte. However, the number of bits in a byte (CHAR_BIT) is implementation-defined (usually 8).

Q27. What will the following code print?

```
#include
int main(){
char c = 'A';
printf("%d", c);
return 0;
}
```

A) A
B) 65
C) 1
D) 0

✔ **Answer: (B)**

Explanation: When a char is printed with '%d', it prints its ASCII value. The ASCII value of 'A' is 65.

Q28. What is the ASCII value of '0' (zero character)?

A) 0
B) 48
C) 32
D) 58

✔ **Answer: (B)**

Explanation: The ASCII value of the character '0' is 48. This is different from the integer value 0. Characters '0'-'9' have ASCII values 48-57.

Q29. What will be the output?

```
#include
int main(){
char c = 'a';
c = c - 32;
printf("%c", c);
return 0;
}
```

A) a
B) A
C) 65
D) 97

✔ **Answer: (B)**

Explanation: ASCII of 'a' = 97. 97 - 32 = 65 = ASCII of 'A'. This is a common technique to convert lowercase to uppercase.

Q30. Which format specifier is used to print a char as its character representation?

A) %d
B) %i
C) %c
D) %s

✔ **Answer: (C)**

Explanation: '%c' is the format specifier for printing a single character. '%d' prints the ASCII integer value, '%s' prints a string (null-terminated char array).

Q31. In C, 'char' may be signed or unsigned depending on the implementation. What does signed char guarantee?

A) Range 0 to 255
B) Range -128 to 127
C) Range -256 to 255
D) Range 0 to 127

✔ **Answer: (B)**

Explanation: A signed char uses 1 byte with the most significant bit as sign. Range = -2^7 to 2^7-1 = -128 to 127.

Q32. What is the output?

```
#include
int main(){
char c = 255;
printf("%d", c);
return 0;
}
```

A) 255
B) -1
C) 0
D) Depends on signedness of char

✔ **Answer: (D)**

Explanation: If char is signed (common on x86), 255 overflows and wraps to -1. If char is unsigned, it prints 255. The result is implementation-defined.

Q33. What character does ASCII value 10 represent?

A) Space
B) Tab
C) Newline
D) Null

✔ **Answer: (C)**

Explanation: ASCII 10 is the newline character ('\n'). ASCII 9 is tab ('\t'), ASCII 32 is space, and ASCII 0 is null ('\0').

Q34. What is the output of this code?

```
#include
int main(){
char c = '5';
int x = c - '0';
printf("%d", x);
```

```
return 0;
}
```

A) 5
B) '5'
C) 53
D) -43

✔ **Answer: (A)**

Explanation: This converts the character '5' to its integer value. '5' has ASCII 53, '0' has ASCII 48. 53 - 48 = 5. This is a standard technique for character-to-digit conversion.

Q35. Which of the following character constants is valid in C?

A) 'AB'
B) "A"
C) 'A'
D) A

✔ **Answer: (C)**

Explanation: 'A' (single quotes) is a valid character constant. "A" is a string literal (array of chars), 'AB' is a multi-character constant (implementation-defined), and A without quotes is an identifier.

Chapter 4: float and double

Floating-Point Types, Precision, and Range

Q36. What is the size of 'float' in C?

A) 2 bytes
B) 4 bytes
C) 8 bytes
D) 10 bytes

✔ **Answer: (B)**

Explanation: A float in C is a single-precision floating-point number occupying 4 bytes (32 bits). It follows the IEEE 754 standard.

Q37. What is the size of 'double' in C?

A) 4 bytes
B) 6 bytes
C) 8 bytes
D) 16 bytes

✔ **Answer: (C)**

Explanation: A double in C is a double-precision floating-point number occupying 8 bytes (64 bits). It has more precision and a wider range than float.

Q38. What format specifier is used for 'float' in printf()?

A) %f
B) %F
C) %lf
D) %e only

✔ **Answer: (A)**

Explanation: '%f' is used for float (and double) in printf(). In scanf(), '%f' is for float and '%lf' is for double. In printf(), both float and double use '%f' due to default argument promotion.

Q39. What is the approximate precision (significant decimal digits) of 'float'?

A) 3-4 digits
B) 6-7 digits
C) 15-16 digits
D) 18-19 digits

✔ **Answer: (B)**

Explanation: Float (32-bit) has approximately 6-7 significant decimal digits of precision. Double has 15-16 digits, and long double has 18-19 digits.

Q40. What is the output?

```
#include
int main(){
printf("%d", sizeof(double));
return 0;
}
```

A) 4
B) 6
C) 8

D) 16

✔ **Answer: (C)**

Explanation: sizeof(double) returns 8 on most systems. Note: using %d with sizeof result is technically incorrect (should use %zu), but on most 32-bit systems it prints 8.

Q41. What is 'long double' in C?

A) Same as double
B) Extended precision floating-point (80-bit or 128-bit)
C) A 16-byte integer
D) Double with long range

✔ **Answer: (B)**

Explanation: 'long double' is an extended-precision floating-point type. On x86 Linux it's 80-bit (10 bytes stored in 12 or 16 bytes). On some systems it equals double.

Q42. Which of the following is NOT a valid floating-point literal?

A) 3.14f
B) 3.14L
C) 3.14
D) 3.14D

✔ **Answer: (D)**

Explanation: '3.14D' is invalid. Valid floating suffixes: 'f' or 'F' for float, 'l' or 'L' for long double. No suffix means double. 'D' is not a valid suffix.

Q43. What is the approximate range of 'float'?

A) ±3.4 x 10^38
B) ±1.7 x 10^308
C) ±3.4 x 10^18
D) ±1.7 x 10^38

✔ **Answer: (A)**

Explanation: Float (IEEE 754 single precision) has a range of approximately ±3.4 × 10^38. Double's range is ±1.7 × 10^308.

Q44. What will be the output?

```
#include
int main(){
float f = 1.1;
double d = 1.1;
if(f == d)
printf("Equal");
else
printf("Not Equal");
return 0;
}
```

A) Equal
B) Not Equal
C) Compile error
D) Runtime error

✔ **Answer: (B)**

Explanation: Due to floating-point representation differences, 1.1 as float and 1.1 as double are not exactly equal. Float has less precision, so the comparison fails. Always use epsilon-based comparison for floats.

Q45. Which header defines FLT_MAX and DBL_MAX?

A) limits.h
B) float.h
C) math.h
D) stdlib.h

✔ **Answer: (B)**

Explanation: defines constants like FLT_MAX, FLT_MIN, DBL_MAX, DBL_MIN, FLT_DIG (precision digits), etc. for floating-point types.

Q46. What does 'FLT_DIG' represent?

A) The size of float in digits
B) The number of significant decimal digits for float
C) The exponent range of float
D) A float comparison function

✔ **Answer: (B)**

Explanation: FLT_DIG represents the number of significant decimal digits that can be represented by a float without loss of information. Typically 6.

Q47. What is the output?

```
#include
int main(){
float x = 1.0/3.0;
printf("%.10f", x);
return 0;
}
```

A) 0.3333333333
B) 0.3333333433 (approximately)
C) 0.3333333333333
D) 0.0

✔ **Answer: (B)**

Explanation: Float has ~6-7 digits of precision. After that, rounding errors appear. 1.0/3.0 as float approximates to about 0.3333333433... when displayed to 10 decimal places.

Chapter 5: sizeof() Operator

Memory Size, sizeof() with Data Types & Expressions

Q48. The sizeof() operator in C returns:

A) The number of elements in an array
B) The memory size in bytes
C) The range of a data type
D) The address of a variable

✔ Answer: (B)

Explanation: sizeof() is a compile-time unary operator that returns the size of its operand in bytes. It can be applied to types, variables, or expressions.

Q49. What is the correct format specifier to print the result of sizeof()?

A) %d
B) %i
C) %zu
D) %u

✔ Answer: (C)

Explanation: sizeof() returns a value of type size_t (an unsigned integer type). The correct format specifier is '%zu'. Using '%d' may produce warnings or incorrect output on 64-bit systems.

Q50. What is the output?

```
#include
int main(){
printf("%zu", sizeof(char));
return 0;
}
```

A) 0
B) 1
C) 2
D) 4

✔ Answer: (B)

Explanation: sizeof(char) is always 1 by definition in C. This is the fundamental unit of measurement.

Q51. What is the output?

```
#include
int main(){
int arr[5];
printf("%zu", sizeof(arr));
return 0;
}
```

A) 5
B) 10
C) 20
D) 40

✔ Answer: (C)

Explanation: sizeof(arr) = number of elements × size of each element = 5 × sizeof(int) = 5 × 4 = 20 bytes.

Q52. What does the following expression evaluate to?

```
sizeof(int) + sizeof(float)
```

A) 2
B) 4
C) 6
D) 8

✔ **Answer: (D)**

Explanation: sizeof(int) = 4, sizeof(float) = 4. So 4 + 4 = 8.

Q53. Is sizeof() evaluated at compile time or runtime?

A) Always at runtime
B) Always at compile time
C) Compile time (mostly), runtime for VLAs in C99
D) Both equally

✔ **Answer: (C)**

Explanation: sizeof() is a compile-time operator for fixed types. For Variable Length Arrays (VLAs) introduced in C99, sizeof() is evaluated at runtime since array size is determined at runtime.

Q54. What is the output?

```
#include
int main(){
int x = 5;
printf("%zu", sizeof(x++));
return 0;
}
```

A) 4
B) 5
C) 6
D) sizeof() causes x to increment

✔ **Answer: (A)**

Explanation: sizeof() does NOT evaluate its operand. x++ is never executed. sizeof(x) = sizeof(int) = 4. x remains 5 after this statement.

Q55. What is sizeof(double) / sizeof(float)?

A) 1
B) 2
C) 4
D) 0.5

✔ **Answer: (B)**

Explanation: sizeof(double) = 8, sizeof(float) = 4. 8/4 = 2 (integer division). This ratio is always 2.

Q56. How do you find the number of elements in an array 'arr' using sizeof()?

A) sizeof(arr) / sizeof(int)
B) sizeof(arr) / sizeof(arr[0])
C) sizeof(arr[0]) / sizeof(arr)
D) sizeof(arr)

✔ **Answer: (B)**

Explanation: sizeof(arr) gives total bytes; sizeof(arr[0]) gives bytes per element. Dividing gives the count. Using arr[0] is preferred over a hardcoded type to avoid errors if the type changes.

Q57. What is the output?

```
#include
struct Student{
int roll;
char name[20];
};
int main(){
printf("%zu", sizeof(struct Student));
return 0;
}
```

A) 21
B) 24
C) 22
D) 20

✔ **Answer: (B)**

Explanation: Due to structure padding/alignment: int roll = 4 bytes, char name[20] = 20 bytes. Total = 24 bytes (the compiler adds padding to align int to a 4-byte boundary). Actual result may vary by compiler.

Q58. sizeof() on a pointer variable returns:

A) Size of pointed-to data type
B) 4 or 8 bytes depending on architecture
C) Always 4 bytes
D) Always 8 bytes

✔ **Answer: (B)**

Explanation: sizeof(pointer) returns the size of the pointer itself, not what it points to. On 32-bit systems it's 4 bytes; on 64-bit systems it's 8 bytes.

Q59. What is the output?

```
#include
int main(){
printf("%zu %zu", sizeof(long), sizeof(long long));
return 0;
}
```

A) 4 8
B) 8 8
C) 4 4
D) Depends on platform

✔ **Answer: (D)**

Explanation: On Linux 64-bit: long=8, long long=8. On Windows 64-bit: long=4, long long=8. On 32-bit: long=4, long long=8. The answer is platform-dependent.

Q60. Can sizeof() be applied to a function?

A) Yes, returns function size
B) No, sizeof() cannot be applied to functions
C) Yes, returns return type size
D) Yes, returns parameter size

✔ **Answer: (B)**

Explanation: sizeof() cannot be applied to functions, incomplete types, or bit-fields. Applying sizeof() to a function name is a compile-time error.

Chapter 6: Type Conversion & Type Casting

Implicit Conversion, Explicit Casting & Promotion Rules

Q61. What is implicit type conversion in C?

A) Manual conversion using cast operator
B) Automatic conversion by the compiler
C) Conversion using library functions
D) Conversion that causes data loss always

✔ Answer: (B)

Explanation: Implicit type conversion (also called type coercion or automatic type conversion) is done automatically by the compiler when operands of different types are involved in an expression.

Q62. What is the output?

```
#include
int main(){
int a = 5, b = 2;
float result = a/b;
printf("%f", result);
return 0;
}
```

A) 2.500000
B) 2.000000
C) 2.5
D) 3.0

✔ Answer: (B)

Explanation: a/b performs integer division (5/2=2). The result 2 is then converted to float: 2.000000. To get 2.5, use: (float)a/b or a/(float)b or 5.0/2.

Q63. What is the output?

```
#include
int main(){
int a = 5, b = 2;
float result = (float)a/b;
printf("%f", result);
return 0;
}
```

A) 2.000000
B) 2.500000
C) 2.5000000
D) 2

✔ Answer: (B)

Explanation: (float)a casts 'a' to float first, making it 5.0. Then 5.0/2 performs floating-point division giving 2.5. printf formats to 6 decimal places by default: 2.500000.

Q64. In C, which implicit conversion rule applies when int and float are used together?

A) float is converted to int
B) int is converted to float
C) Both are converted to double
D) No conversion occurs

✔ **Answer: (B)**

Explanation: When int and float are in the same expression, the int is implicitly promoted to float. This is called 'usual arithmetic conversion'. The result is float.

Q65. What is explicit type casting?

A) Automatic conversion by the compiler
B) Conversion using a cast operator (type) before a variable
C) Conversion using scanf()
D) Conversion using format specifiers

✔ **Answer: (B)**

Explanation: Explicit type casting is done by the programmer using the cast operator: (type)expression. Example: (int)3.14 converts double 3.14 to int 3.

Q66. What is the output?

```
#include
int main(){
float f = 9.7;
int i = (int)f;
printf("%d", i);
return 0;
}
```

A) 10
B) 9
C) 9.7
D) 0

✔ **Answer: (B)**

Explanation: Casting float to int truncates (not rounds) the fractional part. (int)9.7 = 9. The .7 is simply discarded.

Q67. What is the output?

```
#include
int main(){
int a = 'A';
printf("%d", a);
return 0;
}
```

A) A
B) 65
C) 1
D) 0

✔ **Answer: (B)**

Explanation: 'A' is a char with ASCII value 65. When assigned to an int, implicit conversion occurs: int a = 65. Printing with %d shows 65.

Q68. What is the hierarchy of data type promotion in C (from lowest to highest)?

A) int → char → float → double
B) char → int → float → double → long double
C) float → int → double → char
D) char → float → int → double

✔ **Answer: (B)**

Explanation: C's type promotion hierarchy (from lower to higher): char → short → int → long → long long → float → double → long double. Lower types are promoted to higher types in mixed expressions.

Q69. What is 'narrowing conversion'?

A) Converting smaller type to larger type
B) Converting larger type to smaller type (may lose data)
C) Converting int to float
D) Converting char to int

✔ Answer: (B)

Explanation: Narrowing conversion (also called demotion) converts a larger type to a smaller one, potentially losing data. Example: double to float, long to int. This may cause warnings.

Q70. What is the output?

```
#include
int main(){
double d = 3.99;
int i;
i = d;
printf("%d", i);
return 0;
}
```

A) 4
B) 3
C) 3.99
D) Compile error

✔ Answer: (B)

Explanation: Implicit conversion from double to int truncates (not rounds) the fractional part. 3.99 becomes 3. A compiler warning may be issued for implicit narrowing.

Q71. Which of the following is an example of widening conversion?

A) int to char
B) double to float
C) float to double
D) long to int

✔ Answer: (C)

Explanation: Widening conversion (promotion) goes from a smaller type to a larger type without data loss. float to double is widening (4 bytes → 8 bytes, more precision).

Q72. What is the output?

```
#include
int main(){
int x = 300;
char c = x;
printf("%d", c);
return 0;
}
```

A) 300
B) 44
C) -44
D) Depends on signedness of char

✔ Answer: (D)

Explanation: 300 in binary (9 bits) is truncated to 8 bits: 300 % 256 = 44. If char is signed and bit 7 is 0, result is 44. If the result were >127 for signed char, it would be negative. Here 44 is within signed char range, so answer is 44, but the question highlights implementation dependency.

Q73. In the expression '3 + 4.0', what type is the result?

A) int
B) float
C) double
D) long double

✔ **Answer: (C)**

Explanation: 3 is an int, 4.0 is a double literal (no 'f' suffix). The int 3 is promoted to double. The result is of type double.

Q74. What is atoi() used for in C?

A) Converts int to ASCII
B) Converts string to int
C) Converts int to string
D) Converts float to int

✔ **Answer: (B)**

Explanation: atoi() (from) converts a string (const char*) to an integer. Example: atoi("42") returns 42.

Q75. What does the expression (char)300 evaluate to?

A) 300
B) 44
C) Compile error
D) Undefined

✔ **Answer: (B)**

Explanation: 300 mod 256 = 44. When casting to char (1 byte), only the lower 8 bits are kept. 300 in binary = 100101100; lower 8 bits = 00101100 = 44.

Chapter 7: Derived & User-Defined Types

Arrays, Pointers, struct, union, enum, typedef

Q76. Which of the following is a derived data type in C?

A) enum
B) struct
C) pointer
D) union

✔ **Answer: (C)**

Explanation: Pointers are derived data types (derived from basic types). Arrays and functions are also derived types. struct, union, and enum are user-defined types.

Q77. What is a structure in C?

A) A collection of same-type elements
B) A collection of different-type elements under one name
C) A pointer to a function
D) A derived data type only

✔ **Answer: (B)**

Explanation: A structure (struct) is a user-defined data type that groups variables of different data types under a single name. Each member has its own memory location.

Q78. What is the key difference between struct and union?

A) struct members share memory; union members don't
B) union members share the same memory location; struct members have separate memory
C) struct can have functions; union cannot
D) No difference

✔ **Answer: (B)**

Explanation: In a union, all members share the same memory location. The size of a union equals the size of its largest member. In a struct, each member has its own memory.

Q79. What is the size of the union?

```
union Data{
int i;
float f;
char c;
};
```

A) 9 bytes
B) 8 bytes
C) 4 bytes
D) 1 byte

✔ **Answer: (C)**

Explanation: sizeof(union) = size of its largest member. int=4, float=4, char=1. Largest is 4. So sizeof(Data) = 4 bytes.

Q80. What is an enum in C?

A) A set of named integer constants
B) A type alias
C) A derived pointer type

D) A character array

✔ **Answer: (A)**

Explanation: An enum (enumeration) defines a set of named integer constants. By default, the first member = 0, second = 1, etc. You can assign custom values.

Q81. What value does 'WEDNESDAY' have?

```
enum Day{MON, TUE, WED, THU, FRI};
```

A) 1
B) 2
C) 3
D) 4

✔ **Answer: (B)**

Explanation: By default, enum values start at 0. MON=0, TUE=1, WED=2, THU=3, FRI=4. WED (Wednesday) = 2.

Q82. What is 'typedef' used for in C?

A) To define new data types from scratch
B) To create an alias (alternate name) for an existing type
C) To declare global variables
D) To import external types

✔ **Answer: (B)**

Explanation: typedef creates an alias for an existing type. Example: typedef unsigned int uint; Now 'uint' can be used instead of 'unsigned int'. typedef does not create a new type.

Q83. What is a pointer in C?

A) A variable that stores a value
B) A variable that stores the memory address of another variable
C) A function parameter
D) An array of integers

✔ **Answer: (B)**

Explanation: A pointer is a derived data type variable that stores the memory address of another variable. Declared as: int *ptr; where ptr holds the address of an int.

Q84. What is the size of a pointer on a 64-bit system?

A) 2 bytes
B) 4 bytes
C) 8 bytes
D) 16 bytes

✔ **Answer: (C)**

Explanation: On a 64-bit system, all pointer types occupy 8 bytes (64 bits) regardless of what they point to. On 32-bit systems, pointers are 4 bytes.

Q85. What is a void pointer (void*)?

A) A pointer that points to void
B) A generic pointer that can point to any data type
C) A NULL pointer
D) A pointer to a function

✔ **Answer: (B)**

Explanation: A void* (void pointer) is a generic pointer that can hold the address of any data type. It must be cast to a specific type before dereferencing.

Q86. What is the output?

```
#include
typedef int Integer;
int main(){
Integer x = 42;
printf("%d", x);
return 0;
}
```

A) Compile error
B) 42
C) 0
D) Integer

✔ Answer: (B)

Explanation: 'typedef int Integer;' creates 'Integer' as an alias for int. 'Integer x = 42;' is equivalent to 'int x = 42;'. Output is 42.

Q87. In an enum, if you write:

```
enum Color{RED=5, GREEN, BLUE};
What is the value of BLUE?
```

A) 5
B) 6
C) 7
D) 2

✔ Answer: (C)

Explanation: RED=5 (explicitly set). GREEN = RED+1 = 6, BLUE = GREEN+1 = 7. Subsequent members increment by 1 from the previous value.

Chapter 8: Output-Based & Tricky Questions

Predict the Output — Advanced Practice

Q88. What is the output?

```
#include
int main(){
int a = 10, b = 3;
printf("%f", (float)(a/b));
return 0;
}
```

A) 3.333333
B) 3.000000
C) 3.5
D) Compile error

✔ Answer: (B)

Explanation: a/b is integer division = 3. Then (float)3 = 3.0. Cast happens AFTER division. To get 3.333..., write (float)a/b.

Q89. What is the output?

```
#include
int main(){
printf("%d", sizeof('A'));
return 0;
}
```

A) 1
B) 2
C) 4
D) 8

✔ Answer: (C)

Explanation: In C (not C++), a character literal like 'A' has type int, not char! So sizeof('A') = sizeof(int) = 4. This is a common trick question.

Q90. What is the output?

```
#include
int main(){
int x = 010;
printf("%d", x);
return 0;
}
```

A) 10
B) 8
C) 2
D) 0

✔ Answer: (B)

Explanation: 010 is an octal literal (starts with 0). Octal 10 = 1×8 + 0 = 8 in decimal. This catches programmers who accidentally pad integers with zeros.

Q91. What is the output?

```
#include
int main(){
int x = 0x10;
printf("%d", x);
return 0;
}
```

A) 10
B) 16
C) 0
D) 1

✔ **Answer: (B)**

Explanation: 0x10 is a hexadecimal literal. Hex 10 = 1×16 + 0 = 16 in decimal.

Q92. What is the output?

```
#include
int main(){
char c = 'a' + 1;
printf("%c", c);
return 0;
}
```

A) a1
B) b
C) 98
D) a+1

✔ **Answer: (B)**

Explanation: 'a' has ASCII 97. 97+1=98. ASCII 98 = 'b'. Since we use %c, it prints 'b'.

Q93. What is the output?

```
#include
int main(){
int i = 1;
float f = 1;
printf("%d", i == f);
return 0;
}
```

A) 0
B) 1
C) Compile error
D) Undefined

✔ **Answer: (B)**

Explanation: i (int 1) is promoted to float (1.0). 1.0f == 1.0f is true (1). For powers of 2 and small integers, float represents them exactly, so comparison holds.

Q94. What is the output?

```
#include
int main(){
int a = -1;
unsigned int b = 1;
if(a > b)
printf("a > b");
else
printf("a <= b");
```

```
return 0;
}
```

A) a > b
B) a <= b
C) Compile error
D) Undefined

✔ **Answer: (A)**

Explanation: When int and unsigned int are compared, the int is converted to unsigned. -1 as unsigned int = 4294967295 (all bits 1), which is > 1. Counterintuitive but correct.

Q95. What is the output?

```
#include
int main(){
float x = 0.1 + 0.2;
if(x == 0.3)
printf("Equal");
else
printf("Not Equal");
return 0;
}
```

A) Equal
B) Not Equal
C) Compile error
D) Unpredictable

✔ **Answer: (B)**

Explanation: 0.1 + 0.2 in floating-point is not exactly 0.3 due to binary representation errors. This is the classic floating-point precision issue. Always use epsilon comparison: fabs(x-0.3) < 1e-9.

Q96. What is the output?

```
#include
int main(){
int x;
printf("%d", sizeof(x=5));
return 0;
}
```

A) 5
B) 4
C) 0
D) Compile error

✔ **Answer: (B)**

Explanation: sizeof() does NOT evaluate its operand. x=5 is never executed. sizeof(x) = sizeof(int) = 4. x remains uninitialized.

Q97. What is the output?

```
#include
int main(){
double d = 5;
int i = 2;
printf("%f", d/i);
return 0;
}
```

A) 2
B) 2.000000
C) 2.500000
D) 2.5

✔ Answer: (C)

Explanation: d is double (5.0), i is int (2). i is promoted to double. 5.0/2.0 = 2.5. printf with %f shows 2.500000 (6 decimal places by default).

Q98. What is the output?

```
#include
int main(){
char a = 200;
printf("%d", a);
return 0;
}
```

A) 200
B) -56
C) 56
D) Depends on char signedness

✔ Answer: (D)

Explanation: If char is signed: 200 > 127, so it wraps. 200 - 256 = -56. If char is unsigned: 200. The behavior is implementation-defined.

Q99. What is the output?

```
#include
int main(){
int x = 5.9;
printf("%d", x);
return 0;
}
```

A) 6
B) 5
C) 5.9
D) Compile error

✔ Answer: (B)

Explanation: Assigning double 5.9 to int truncates (not rounds) to 5. printf prints 5.

Q100. What is the output?

```
#include
int main(){
printf("%d %d %d",
sizeof(int), sizeof(float),
sizeof(double));
return 0;
}
```

A) 2 4 8
B) 4 4 8
C) 4 8 16
D) 2 4 4

✔ Answer: (B)

Explanation: On most 32/64-bit systems: sizeof(int)=4, sizeof(float)=4, sizeof(double)=8. Note: using %d for size_t is technically a warning but works on 32-bit systems.

Q101. What is the output?

```
#include
int main(){
int i = 3;
float j = 3.0;
printf("%d", i == j);
return 0;
}
```

A) 0
B) 1
C) Error
D) -1

✔ Answer: (B)

Explanation: i (int 3) is promoted to float (3.0f). 3.0f == 3.0f is true → 1. Powers of 2 and small integers are represented exactly in IEEE 754 float.

Q102. What is the output?

```
#include
int main(){
unsigned char c = 0;
c--;
printf("%d", c);
return 0;
}
```

A) -1
B) 255
C) 0
D) Undefined behavior

✔ Answer: (B)

Explanation: Unsigned char wraps around. 0 - 1 wraps to 255 (unsigned wraparound is well-defined in C, unlike signed overflow).

Chapter 9: Range, Size Reference & Type Modifiers

Complete Reference Questions for Exam Preparation

Q103. What is the range of 'double'?

A) ±3.4 × 10^38
B) ±1.7 × 10^308
C) ±9.2 × 10^18
D) ±3.4 × 10^18

✔ **Answer: (B)**

Explanation: double (IEEE 754 double precision, 64-bit) has a range of approximately ±1.7 × 10^308 with about 15-16 significant decimal digits.

Q104. Which type modifier cannot be applied to float or double?

A) long
B) short
C) signed
D) Both B and C

✔ **Answer: (D)**

Explanation: 'short' and 'signed' cannot be applied to float or double. 'long' can (long double is valid). Floating-point types are always signed and have no short variant.

Q105. What is the approximate size of 'long double' on most Linux x86 systems?

A) 8 bytes
B) 10 bytes
C) 12 bytes
D) 16 bytes

✔ **Answer: (C)**

Explanation: On Linux x86, long double uses 80-bit extended precision but is stored in 12 bytes (32-bit) or 16 bytes (64-bit) due to alignment. sizeof(long double) is often 12 or 16.

Q106. The format specifier for 'long double' in printf() is:

A) %lf
B) %Lf
C) %llf
D) %LD

✔ **Answer: (B)**

Explanation: '%Lf' (capital L) is the format specifier for long double. '%lf' is for double in scanf() (and also accepted in modern printf for double).

Q107. Which C standard introduced 'long long int'?

A) C89/ANSI C
B) C95
C) C99
D) C11

✔ **Answer: (C)**

Explanation: 'long long int' was introduced in the C99 standard (ISO/IEC 9899:1999). It is guaranteed to be at least 64 bits.

Q108. What is CHAR_BIT defined as?

A) 4
B) 8
C) 16
D) Number of bits in a char, typically 8

✔ **Answer: (D)**

Explanation: CHAR_BIT (defined in) is the number of bits in a char. On virtually all modern systems it is 8, but the C standard allows other values for specialized hardware.

Q109. What is the value of INT_MAX on a 32-bit system?

A) 32767
B) 65535
C) 2147483647
D) 4294967295

✔ **Answer: (C)**

Explanation: INT_MAX = 2^31 - 1 = 2,147,483,647 for a 32-bit signed int. This is defined in .

Q110. What does 'ULONG_MAX' represent?

A) Maximum value of unsigned long
B) Maximum value of unsigned long long
C) Maximum value of union long
D) Undefined limit constant

✔ **Answer: (A)**

Explanation: ULONG_MAX is the maximum value of unsigned long. On 32-bit systems it is 4,294,967,295; on 64-bit Linux it is 18,446,744,073,709,551,615.

Q111. When would you prefer 'double' over 'float' in C?

A) When memory is limited
B) When higher precision calculations are needed
C) When working with characters
D) When using integer arithmetic

✔ **Answer: (B)**

Explanation: double provides ~15-16 significant digits vs float's ~6-7. Use double for scientific calculations, financial computations, or any situation requiring higher precision.

Q112. Which of the following has the highest precision in C?

A) float
B) double
C) long double
D) int

✔ **Answer: (C)**

Explanation: long double has the highest precision among floating-point types in C. The order from lowest to highest: float < double < long double.

Q113. What is the minimum size guaranteed for 'long int' by the C standard?

A) 16 bits
B) 32 bits
C) 64 bits
D) 128 bits

✔ **Answer: (B)**

Explanation: The C standard guarantees long int is at least 32 bits (4 bytes). On 64-bit Linux (LP64), long is 64 bits. On Windows (LLP64), long remains 32 bits.

Q114. What does 'DBL_EPSILON' represent?

A) The smallest positive double
B) The smallest difference between 1.0 and the next representable double
C) The maximum double value
D) Double precision exponent limit

✔ **Answer: (B)**

Explanation: DBL_EPSILON (defined in) is the smallest value such that 1.0 + DBL_EPSILON != 1.0. It is approximately 2.22045 × 10^-16. Used in floating-point comparisons.

Q115. The expression 'sizeof(void)' in C is:

A) 0
B) 1
C) Undefined
D) Compile error in standard C

✔ **Answer: (D)**

Explanation: sizeof(void) is undefined in standard C (C90/C99/C11). GCC as an extension returns 1 for sizeof(void), but this is non-standard and should not be relied upon.

Q116. What is the output?

```
#include
int main(){
int a = 5;
printf("%zu", sizeof(a+3.0));
return 0;
}
```

A) 4
B) 6
C) 8
D) Depends on system

✔ **Answer: (C)**

Explanation: a (int) + 3.0 (double literal) → result is double. sizeof(double) = 8. sizeof() sees the type of the expression, not the value.

Q117. Which of the following correctly represents the memory layout priority (largest to smallest)?

A) long double > double > float > long > int > short > char
B) char > short > int > long > float > double
C) int > float > double > char
D) float > int > char > double

✔ **Answer: (A)**

Explanation: In terms of size: long double (10-16 bytes) > double (8) > float (4) = long* = int (4) > short (2) > char (1). *long varies by platform.

Exam Tips & Key Formulas

Last-Minute Revision Points for OAVS TGT Exam

sizeof() Key Facts

• sizeof(char) = 1 (always, by definition)
• sizeof(int) = 4 (typical 32/64-bit)
• sizeof(float) = 4, sizeof(double) = 8
• sizeof(pointer) = 4 (32-bit) or 8 (64-bit)
• sizeof() does NOT evaluate its operand
• In C, sizeof('A') = sizeof(int) = 4 (char literals are int!)

Type Conversion Rules

• Widening: char→short→int→long→float→double→long double
• Casting int to float may lose precision for large values
• (int)float truncates (does NOT round)
• Signed integer overflow = Undefined Behavior
• Unsigned integer overflow = well-defined wrap-around
• int/int = integer division (e.g., 5/2 = 2, not 2.5)

Format Specifiers

• %d : int • %u : unsigned int
• %ld : long • %lld : long long
• %f : float/double (printf) • %lf : double (scanf)
• %Lf : long double • %c : char • %s : string
• %o : octal • %x : hex • %p : pointer • %zu : size_t

Range Formulas

• Signed N-bit: -(2^(N-1)) to 2^(N-1)-1
• Unsigned N-bit: 0 to 2^N - 1
• signed char (8-bit): -128 to 127
• unsigned char (8-bit): 0 to 255
• signed short (16-bit): -32768 to 32767
• signed int (32-bit): -2,147,483,648 to 2,147,483,647

User-Defined Types

• struct: each member has separate memory
• union: all members share same memory (size = largest member)
• enum: named integer constants, default starts at 0
• typedef: creates alias for existing type (not a new type)
• Structures may have padding for alignment

Common Tricky Points

• 010 = octal 8 (decimal), 0x10 = hex 16 (decimal)
• 'unsigned float' is INVALID in C
• 'string' is NOT a data type in C
• unsigned int compared with negative int: int converts to unsigned
• 0.1 + 0.2 ≠ 0.3 due to floating-point representation

Practice Test 1 — Quick Fire (20 Questions)

Time: 20 Minutes | Marks: 20 | Select the best answer

Q1. Which data type has the smallest size in C?

A) int
B) float
C) char
D) short

Q2. sizeof(float) = ?

A) 2
B) 4
C) 8
D) 16

Q3. Format specifier for double in printf?

A) %f
B) %lf
C) %Lf
D) %d

Q4. Range of signed char?

A) 0–255
B) -128–127
C) -256–255
D) 0–127

Q5. Which is NOT a basic data type?

A) int
B) pointer
C) float
D) char

Q6. int x = 'Z'; printf("%d",x); Output?

A) Z
B) 90
C) 26
D) 0

Q7. sizeof(int)==sizeof(float) is usually:

A) True
B) False
C) Undefined
D) Error

Q8. 'unsigned' can be applied to:

A) float
B) double
C) int
D) All

Q9. atoi() converts:

A) int to string
B) string to int
C) char to int
D) float to int

Q10. In C, 'string' is a:

A) Basic type
B) char array
C) keyword
D) Built-in type

Q11. int a=7,b=2; float c=a/b; c=?

A) 3.5
B) 3.0
C) 3
D) 3.500000

Q12. (int)3.9 = ?

A) 4
B) 3
C) 3.9
D) Error

Q13. sizeof('A') in C = ?

A) 1
B) 2
C) 4
D) 8

Q14. What is the value of 010 in decimal?

A) 10
B) 8
C) 2
D) 1

Q15. Which header for INT_MAX?

A) math.h
B) stdio.h
C) limits.h
D) stdlib.h

Q16. Format specifier for long long int?

A) %ld
B) %lld
C) %lldi
D) %Li

Q17. Largest floating point type in C?

A) float
B) double
C) long double

D) float128

Q18. User-defined data type?

A) int
B) float
C) struct
D) pointer

Q19. sizeof(char*) on 64-bit?

A) 1
B) 4
C) 8
D) 2

Q20. enum{A,B,C}; value of B?

A) 0
B) 1
C) 2
D) 3

Practice Test 2 — Conceptual & Output (20 Questions)

Time: 25 Minutes | Marks: 20 | Include output prediction questions

Q21. int x=-1; unsigned int y=0; (x>y)?

A) True
B) False
C) Error
D) Undefined

Q22. Which is widening conversion?

A) double→float
B) float→int
C) int→double
D) long→char

Q23. FLT_MAX is defined in:

A) limits.h
B) float.h
C) math.h
D) stdio.h

Q24. char c=0; c--; printf("%d",c); Output (unsigned char)?

A) -1
B) 255
C) 0
D) 256

Q25. sizeof(double)/sizeof(float) = ?

A) 1
B) 2
C) 4
D) 0.5

Q26. int arr[10]; sizeof(arr)/sizeof(arr[0]) = ?

A) 4
B) 10
C) 40
D) 5

Q27. long double format specifier in printf?

A) %lf
B) %Lf
C) %llf
D) %LD

Q28. Does sizeof() evaluate x++ ?

A) Yes
B) No
C) Sometimes
D) Depends

Q29. 0.1+0.2==0.3 in C is usually:

A) True
B) False
C) Always true
D) Compile error

Q30. Which type for highest integer range?

A) int
B) long
C) long long
D) unsigned long long

Q31. int a=5; float b=a; is:

A) Explicit cast
B) Implicit (widening)
C) Narrowing
D) Illegal

Q32. sizeof(void) in standard C?

A) 0
B) 1
C) Error/Undefined
D) 4

Q33. short int range (signed)?

A) -128–127
B) 0–65535
C) -32768–32767
D) -256–255

Q34. ULONG_MAX on 32-bit?

A) 2147483647
B) 4294967295
C) 65535
D) 9223372036854775807

Q35. int x=300; char c=x; c=? (signed char)

A) 300
B) 44
C) -44
D) 0

Q36. Derived types in C include:

A) struct
B) enum
C) arrays & pointers
D) typedef

Q37. float f=9.7; int i=(int)f; i=?

A) 10
B) 9
C) 9.7

D) Error

Q38. typedef creates:

A) New type
B) Type alias
C) Global variable
D) Macro

Q39. Union size equals:

A) Sum of all members
B) Size of smallest member
C) Size of largest member
D) 4 bytes always

Q40. int a=2147483647; a+1 in C is:

A) 2147483648
B) -2147483648
C) 0
D) Undefined behavior

ANSWER KEYS

Practice Test 1 — Quick Fire (20 Questions)

Q1: C Q2: B Q3: A Q4: B Q5: B
Q6: B Q7: A Q8: C Q9: B Q10: B
Q11: B Q12: B Q13: C Q14: B Q15: C
Q16: B Q17: C Q18: C Q19: C Q20: B

Practice Test 2 — Conceptual & Output (20 Questions)

Q21: A Q22: C Q23: B Q24: B Q25: B
Q26: B Q27: B Q28: B Q29: B Q30: D
Q31: B Q32: C Q33: C Q34: B Q35: B
Q36: C Q37: B Q38: B Q39: C Q40: D